

PROJECT REPORT

Student Attendance Management System

A beginner-friendly Python project for recording student attendance and calculating attendance percentages.

Project Title	Student Attendance Management System
Author	Ameya D. Giradkar
Technology	Python
Project Type	Academic / Beginner-Level Python Project

Prepared according to the project submission guidelines provided for the VITyarthi flipped-course evaluation.

1. Introduction

The Student Attendance Management System is a simple Python-based project designed to record student attendance over a selected number of days and calculate the attendance percentage for each student. The project applies basic programming concepts to a real-world educational problem.

The current implementation uses a dictionary to store student names and their present-day counts, a set to prevent duplicate attendance entries for the same day, loops to process students and days, and conditional statements to check whether an entered student exists.

2. Problem Statement

Managing student attendance manually can take time and may result in calculation errors. When attendance is recorded for several students over several days, calculating the number of present days and the final percentage for every student can become repetitive.

This project provides a simple computerized solution in which the user enters the students, the number of attendance days, and the students present on each day. The system then calculates and displays the attendance result.

3. Objectives

- Enter and store the names of students.
- Enter the total number of attendance days.
- Record the students who are present on each day.
- Count the total present days for each student.
- Calculate attendance percentage automatically.
- Display a clear final attendance result.
- Demonstrate practical use of basic Python data structures and control statements.

4. Functional Requirements

4.1 Student Management

The user enters the number of students and the name of each student. Student names are stored in a dictionary, with the initial attendance count set to zero.

4.2 Attendance Management

For each attendance day, the user enters the number of students present and then enters their names. Only names already stored in the student dictionary are accepted for attendance counting.

4.3 Attendance Reporting

After all days are processed, the system displays each student's name, present days, total days, and attendance percentage.

5. Non-Functional Requirements

Requirement	Description
-------------	-------------

Usability	The program uses simple text prompts and a straightforward sequence of inputs.
Reliability	Attendance percentage is calculated automatically using the recorded counts.
Error Handling	Unknown student names are identified instead of being added to attendance.
Maintainability	The implementation uses simple Python structures and control statements that are easy to understand

6. System Architecture

The current project follows a simple input-processing-output architecture. It does not use an external database or web framework.

INPUT	PROCESSING	OUTPUT
Student names Number of days Daily attendance	Dictionary stores counts Set prevents duplicate daily entries Percentage calculation	Attendance result Present days Attendance percentage

7. Workflow Diagram

1	Start
↓	
2	Enter number of students and names
↓	
3	Enter total number of days
↓	
4	For each day, enter present students
↓	
5	Update present-day counts
↓	
6	Calculate attendance percentage
↓	
7	Display attendance result

8. Use Case Description

Actor	Use Case	Description
User	Add students	Enter student names before attendance is recorded.
User	Record attendance	Enter the students present for each day.
System	Calculate attendance	Count present days and calculate percentage.
User	View result	See the final attendance details for each student.

9. Data / Storage Design

The current implementation stores data temporarily in memory while the program is running. A Python dictionary named **students** maps each student name to the number of days that student has been present.

Example logical structure: **students = {"Rahul": 52, "Priya": 45, "Amit": 58}**. A temporary set named **present** stores the students marked present for the current day.

Because the current code does not save information to a database or file, attendance data is lost when the program ends. Persistent storage can be considered as a future enhancement.

10. Design Decisions and Rationale

Decision	Reason
Dictionary for students	Provides a simple mapping between a student's name and present-day count.
Set for daily attendance	Prevents the same student from being counted more than once on a day.
Loops	Allow the same operations to be repeated for multiple students and days.
Simple console interface	Keeps the project understandable for a beginner and avoids unnecessary complexity.
Automatic percentage calculation	Reduces repetitive manual calculations and calculation errors.

11. Implementation Details

11.1 Student Initialization

The program first creates an empty dictionary and asks the user for the number of students. Each entered student name is stored with an initial attendance value of zero.

11.2 Daily Attendance

The program repeats the attendance process for each day. A set is created for that day, and each entered student name is checked against the student dictionary.

11.3 Attendance Calculation

For every valid present student, the corresponding dictionary value is increased by one. At the end, the percentage is calculated using:

```
Attendance Percentage = (Present Days / Total Days) × 100
```

12. Important Code Logic

```
present = set()

number_present = int(input("Enter number of students present: "))

for i in range(number_present):
    name = input("Enter name of present student: ")

    if name in students:
        present.add(name)
    else:
        print("Student not found.")

for name in present:
    students[name] += 1
```

This section demonstrates the core attendance-recording logic: checking a student name, adding valid names to a set, and updating the attendance count.

13. Sample Results

For example, if the program records 60 total days and the following students are present for the listed number of days, the calculated results are:

Name	Present Days	Total Days	Attendance
Rahul	52	60	86.67%
Priya	45	60	75.00%
Amit	58	60	96.67%

Actual screenshots can be added to the final report after running the program in the development environment.

14. Testing Approach

Test Case	Input / Condition	Expected Result
Valid student	Name exists in student list	Student is counted as present.
Unknown student	Name not in student list	Program displays 'Student not found.'
Duplicate daily entry	Same valid name entered twice	Set keeps one occurrence for that day.
Multiple days	Attendance recorded over several days	Present count increases correctly.
Percentage	Present days and total days available	Correct percentage is displayed.

15. Challenges Faced

- Keeping attendance counts correct across multiple days.
- Handling student names that are not part of the registered student list.
- Preventing duplicate attendance entries on the same day.
- Keeping the implementation simple enough for a beginner while still providing multiple project functions.

16. Learnings and Key Takeaways

- Using dictionaries to store and update related data.
- Using sets to avoid duplicate values.
- Using loops for repeated tasks.
- Using conditions for input checking.
- Applying arithmetic formulas in a practical project.
- Designing a simple input-processing-output workflow.

17. Future Enhancements

- Add permanent storage using a file or database.
- Add date-wise attendance records.
- Add monthly and subject-wise reports.
- Add a graphical user interface.
- Export attendance reports to PDF or Excel.
- Add login and role-based access.
- Add more detailed attendance analytics.

18. Conclusion

The Student Attendance Management System demonstrates how basic Python programming concepts can be applied to a practical educational problem. The current program records attendance over multiple days, prevents duplicate daily entries, counts present days, and calculates attendance percentages.

The project is intentionally simple and beginner-friendly while providing a clear foundation for future improvements such as persistent storage, graphical interfaces, and advanced reporting.

19. References

- VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines (uploaded project guideline document).
- Python programming concepts used in the project: dictionaries, sets, loops, conditional statements, input/output, and arithmetic operations.

20. Appendix: Current Program

```
students = {}

n = int(input("Enter number of students: "))

for i in range(n):
    name = input("Enter student name: ")
    students[name] = 0

days = int(input("\nEnter number of days: "))

for day in range(1, days + 1):
    print("\nDay", day)

    present = set()

    number_present = int(input("Enter number of students present: "))

    for i in range(number_present):
        name = input("Enter name of present student: ")

        if name in students:
            present.add(name)
        else:
            print("Student not found.")

    for name in present:
        students[name] += 1

print("\n----- ATTENDANCE RESULT -----")

for name in students:
```

```
present_days = students[name]
percentage = (present_days / days) * 100
print("\nName:", name)
print("Present Days:", present_days)
print("Total Days:", days)
print("Attendance:", round(percentage, 2), "%")
```